

Relazione - Versione 3: Preprocessing Agent

Introduzione

La Versione 3 estende il progetto Building Agentic AutoML introducendo esperimenti automatici di preprocessing.

L'agente continua a ricevere dataset e target, rilevare il task e i tipi di feature, ispezionare il dataset, addestrare un baseline LightGBM, valutarlo e restituire uno stato strutturato. La nuova capacità consiste nel provare più strategie di preprocessing, valutarle con lo stesso protocollo di validazione e conservare il risultato migliore.

Questa versione si concentra sul seguente flusso:

```
Dataset + target
-> Rilevare task e feature
-> Ispezionare il dataset
-> Provare strategie di preprocessing
-> Valutare ogni strategia
-> Selezionare il risultato migliore
-> Restituire stato + esperimenti
```

L'architettura rimane volutamente semplice: una sola famiglia di modelli, un train-validation split, due strategie di preprocessing e nessuna ottimizzazione. La Versione 3 aggiunge sperimentazione sul preprocessing preservando il pipeline precedente.

Obiettivo della Versione 3

L'obiettivo della Versione 3 è permettere all'agente di agire sulle scelte di preparazione dei dati invece di utilizzare un unico passaggio fisso.

L'agente deve ora essere in grado di:

- caricare dataset e target e rilevare il task;
- identificare feature numeriche e categoriche;
- ispezionare dimensioni del dataset, comportamento del target, dtype, valori mancanti e cardinalità;
- provare un piccolo insieme di strategie di preprocessing;
- apprendere i parametri di preprocessing soltanto dai dati di training;
- valutare le strategie con lo stesso modello e lo stesso validation split;
- selezionare il miglior risultato di preprocessing e salvare gli esperimenti nello stato finale.

Adult Income rimane l'esempio principale di classificazione, così che l'unico nuovo concetto introdotto in questa versione sia il preprocessing.

Architettura

Il flusso della Versione 3 è:

```
Dataset + Target -> Task/Features -> Data Inspection -> Preparation
-> Preprocessing Experiments -> Model/Metric -> Evaluation -> Best Result -> State
```

La funzione agent continua a coordinare direttamente l'intero flusso. Il nuovo ciclo di esperimenti di preprocessing viene aggiunto senza introdurre nuove famiglie di modelli, cross-validation, ricerca degli iperparametri o planner.

Rilevamento del task

Il rilevamento del task rimane invariato rispetto alla Versione 2. L'agente usa il numero di valori unici del target come regola semplice: al massimo 20 valori unici significa classificazione; altrimenti il task viene considerato regressione.

```
def detect_task(df, target):  
    n_unique = df[target].nunique()  
    if n_unique <= 20:  
        return "classification"  
    return "regression"
```

Per il target income di Adult Income il risultato è "classification".

Rilevamento delle feature

Anche il rilevamento delle feature rimane invariato. Per Adult Income l'agente rileva 6 feature numeriche e 8 categoriche.

```
numerical = ['age', 'fnlwgt', 'education_num',  
             'capital_gain', 'capital_loss', 'hours_per_week']  
  
categorical = ['workclass', 'education', 'marital_status', 'occupation',  
              'relationship', 'race', 'sex', 'native_country']
```

Data Inspection

Il Data Inspector introdotto nella Versione 2 viene mantenuto invariato. Per Adult Income rileva 48.842 righe e 15 colonne, due classi del target, dtype delle feature, valori mancanti e cardinalità.

I principali valori mancanti restano `workclass = 2799`, `occupation = 2809` e `native_country = 857`. L'inspection rimane descrittiva: viene salvata nello stato ma non decide quali strategie di preprocessing sono disponibili.

Preprocessing

Questa è la nuova capacità introdotta nella Versione 3. L'agente confronta due strategie semplici:

```
native -> mantiene i missing e li lascia gestire a LightGBM  
impute -> mediana per le numeriche, moda per le categoriche
```

Il train-validation split viene creato prima del preprocessing. Mediane, mode e vocabolari categorici vengono quindi appresi soltanto dai dati di training, evitando leakage dal validation set.

Le colonne categoriche vengono convertite usando un `pandas.CategoricalDtype` costruito dalle categorie osservate nel training set, così training e validation condividono lo stesso vocabolario categorico.

Modello baseline e selezione della metrica

La scelta di modello e metrica rimane invariata. La classificazione usa LGBMClassifier con ROC AUC, mentre la regressione usa LGBMRegressor con RMSE. Entrambi i modelli utilizzano random_state=42.

```
classification -> LGBMClassifier + ROC AUC
regression      -> LGBMRegressor + RMSE
```

Addestramento e valutazione

Ogni strategia di preprocessing viene valutata con la stessa famiglia di modelli e lo stesso train-validation split. Lo split usa test_size=0.2 e random_state=42; per la classificazione viene usata anche la stratificazione.

Su Adult Income il notebook ottiene:

```
native -> ROC AUC = 0.9311658417981501
impute -> ROC AUC = 0.930687503244851
best_preprocessing = 'native'
```

Poiché ROC AUC deve essere massimizzata, viene selezionata la strategia native. Il risultato mostra anche che il preprocessing viene trattato come un esperimento: non si assume automaticamente che l'imputazione sia migliore.

Agent

Nella Versione 3, l'Agent esegue le seguenti operazioni:

1. carica dataset e target
2. rileva task e tipi di feature
3. ispeziona il dataset
4. prepara X e y
5. seleziona la metrica
6. prova, addestra e valuta ogni strategia di preprocessing
7. seleziona il miglior esperimento
8. restituisce lo stato finale

Uno stato semplificato contiene ora:

```
{
  "task": "classification",
  "model": "LGBMClassifier",
  "metric": "roc_auc",
  "experiments": [
    {"preprocessing": "native", "score": 0.9311658417981501},
    {"preprocessing": "impute", "score": 0.930687503244851}
  ],
  "best_preprocessing": "native",
  "score": 0.9311658417981501
}
```

Lo stato registra quindi sia la decisione finale sia le evidenze utilizzate per confrontare le strategie di preprocessing disponibili.

Test di regressione

Lo stesso agente viene testato su simple_regression.csv. Rileva automaticamente la regressione, usa RMSE e valuta le stesse due strategie di preprocessing.

```
native -> RMSE = 1.8760451113860066
impute -> RMSE = 1.8776921187920332
best_preprocessing = 'native'
```

Poiché RMSE deve essere minimizzato, viene nuovamente selezionato native. Questo conferma che la logica di selezione degli esperimenti si adatta correttamente alla direzione delle due metriche.

Cosa insegna la Versione 3

La Versione 3 introduce un'idea fondamentale dell'AutoML: le scelte di preparazione dei dati dovrebbero essere valutate empiricamente invece di essere applicate come assunzioni fisse.

L'idea principale diventa:

osservare -> ispezionare -> provare -> valutare -> confrontare -> selezionare -> restituire

L'agente conserva ora una piccola storia degli esperimenti e seleziona il miglior risultato di preprocessing. È la prima versione in cui lo stato registra tentativi alternativi invece di un solo training finale.

Il pattern evolutivo rimane esplicito: la Versione 2 ha imparato a osservare il dataset; la Versione 3 aggiunge un ciclo minimo di azione sul preprocessing mantenendo invariati selezione del modello e strategia di validazione.

Limitazioni

La Versione 3 mantiene alcune limitazioni intenzionali:

- il rilevamento del task usa ancora soltanto il numero di valori unici del target;
- vengono confrontate soltanto due strategie di preprocessing: gestione nativa e imputazione semplice;
- l'imputazione numerica usa soltanto la mediana e quella categorica soltanto il valore più frequente;
- il Data Inspector non decide ancora quali strategie di preprocessing provare;
- non vengono esplorati scaling, alternative di encoding, trattamento degli outlier o feature engineering;
- viene utilizzata una sola famiglia di modelli, LightGBM;
- la valutazione usa ancora un solo train-validation split e non esiste cross-validation;
- non esistono ancora ottimizzazione degli iperparametri, planner o architettura multi-agent.

Queste limitazioni sono deliberate. Lo scopo della Versione 3 è la sperimentazione sul preprocessing, non la selezione del modello o l'ottimizzazione.

Conclusione

La Versione 3 estende Building Agentic AutoML con un piccolo ciclo di esperimenti di preprocessing, mantenendo la semplicità delle versioni precedenti.

L'agente può ora confrontare la gestione nativa dei missing con una semplice imputazione, apprendere i parametri di preprocessing soltanto dai dati di training, valutare entrambe le scelte con lo stesso protocollo, salvare gli esperimenti e restituire la strategia migliore.

La lezione principale è che un agente AutoML non dovrebbe soltanto ispezionare i dati, ma anche provare preparazioni alternative e conservare l'evidenza alla base della propria decisione. Questo prepara il progetto alla Versione 4, dove il prossimo nuovo concetto potrà essere la selezione del modello.